

EfficientML.ai Lab 5 Report

Name: Yinghou Wang

1. Loop Unrolling (20pt): Please fill in the starter code in *kernel/template/loop_unrolling.cc* to implement loop unrolling and run the `./evaluate.sh loop_unrolling` to evaluate performance improvement.
 - a. Please copy and paste your implementation in *kernel/template/loop_unrolling.cc*: (15pt)

```
int intermediate_sum0 = 0, intermediate_sum1 = 0, intermediate_sum2
= 0, intermediate_sum3 = 0;
for (int qj = 0; qj < 16; qj++) {
    // TODO: decode a packed byte into two int8 in the range of (-8,
7)

    uint8_t packed_int4_0 = w0_int4[qj];
    uint8_t packed_int4_1 = w1_int4[qj];
    uint8_t packed_int4_2 = w2_int4[qj];
    uint8_t packed_int4_3 = w3_int4[qj];

    signed char w_de_0_0 = (packed_int4_0 & 0x0F) - 8.0;
    signed char w_de_0_16 = (packed_int4_0 >> 4) - 8.0;
    signed char w_de_1_0 = (packed_int4_1 & 0x0F) - 8.0;
    signed char w_de_1_16 = (packed_int4_1 >> 4) - 8.0;
    signed char w_de_2_0 = (packed_int4_2 & 0x0F) - 8.0;
    signed char w_de_2_16 = (packed_int4_2 >> 4) - 8.0;
    signed char w_de_3_0 = (packed_int4_3 & 0x0F) - 8.0;
    signed char w_de_3_16 = (packed_int4_3 >> 4) - 8.0;

    // TODO: int8 multiply and accumulate operation
    intermediate_sum0 += a_int8[qj] * w_de_0_0;
    intermediate_sum0 += a_int8[qj+16] * w_de_0_16;
    intermediate_sum1 += a_int8[qj] * w_de_1_0;
    intermediate_sum1 += a_int8[qj+16] * w_de_1_16;
    intermediate_sum2 += a_int8[qj] * w_de_2_0;
    intermediate_sum2 += a_int8[qj+16] * w_de_2_16;
    intermediate_sum3 += a_int8[qj] * w_de_3_0;
    intermediate_sum3 += a_int8[qj+16] * w_de_3_16;
}

// dequantize the sum into floating point
acc0 += (float)intermediate_sum0 * s_a * s_w0;
acc1 += (float)intermediate_sum1 * s_a * s_w1;
acc2 += (float)intermediate_sum2 * s_a * s_w2;
```

```

        acc3 += (float)intermediate_sum3 * s_a * s_w3;
        ch += block_size;
    }
    ... ..

```

- b. How does the performance in GOPs, achieved through loop unrolling on your computer, compare to the reference implementation? Please explain the performance difference. (5pt)

----- Sanity check of reference implementation: Passed! -----

Section, Total time(ms), Average time(ms), Count, GOPs
 reference, 3077.961914, 307.795990, 10, 0.851680

----- Sanity check of loop_unrolling implementation: Passed! -----

Section, Total time(ms), Average time(ms), Count, GOPs
 loop_unrolling, 2374.064941, 237.406006, 10, 1.104199

The loop_unrolling achieves an optimization on GOP by increasing 1.2965x, and average time decreasing by 0.2287x.

This is because loop unrolling can reduce branching overhead by replicating the loop body a number of times that helps speeding up.

2. Multithreading (20pt): Please fill in the starter code in *kernel/template/multithreading.cc* to implement multithreading and run the `./evaluate.sh multithreading`` to evaluate performance improvement.

- a. Please copy and paste your implementation in *kernel/template/multithreading.cc*: (15pt)

```

... ..
namespace matmul {
void MatmulOperator::mat_mul_multithreading(struct matmul_params* params) {
    const struct matrix *A = &params->A, *B = &params->B, *C = &params->C;
    const int block_size = params->block_size;

    quantize_fp32_to_int8(A->data_ptr, A->int8_data_ptr, params->A_scales,
A->row * A->column, block_size);

    int m = C->row, n = C->column, k = A->column;

```

```

const int num_thread = 4;
pthread_t thread_pool[num_thread];
struct multithreading_thread_args threads_args[num_thread];

// Thread creation
for (int j = 0; j < num_thread; j++) {
    threads_args[j].start = j * (n / num_thread);
    threads_args[j].end = (j == num_thread - 1) ? n : (j + 1) * (n /
num_thread); // Ensure last thread covers all remaining
    threads_args[j].params = params; // Pass the params pointer directly
    pthread_create(&thread_pool[j], NULL, multithreading_worker_func,
&threads_args[j]);
}
// Join threads
for (int j = 0; j < num_thread; j++)
{
    pthread_join(thread_pool[j], NULL);
}
};
}

```

- b. How does the performance in GOPs, achieved through multithreading on your computer, compare to the reference implementation? Please explain the performance difference. (5pt)

----- Sanity check of reference implementation: Passed! -----

Section, Total time(ms), Average time(ms), Count, GOPs
reference, 3077.961914, 307.795990, 10, 0.851680

----- Sanity check of multithreading implementation: Passed! -----

Section, Total time(ms), Average time(ms), Count, GOPs
multithreading, 768.650024, 76.864998, 10, 3.410447

Multithreading method has an increase in performance by almost 4x compared to the reference implementation. The time consumption decreases by about 0.2497x. It is the concurrent execution of multiple threads within a single process. Different threads can run on separate CPU cores, execute tasks simultaneously, improving performance and allowing parallelism.

3. SIMD Programming (20pt): Please fill in the starter code in *kernel/template/simd_programming.cc* to implement SIMD programming and run the `./evaluate.sh simd_programming` to evaluate the performance improvement.
- a. Please copy and paste your implementation in *kernel/template/simd_programming.cc*: (15pt)

```
// TODO: decode the lower and upper half of the weights as int8x16_t
// Hint:
// (1) use `vandq_u8` with the mask_low4bit to get the lower half
// (2) use `vshrq_n_u8` to right shift 4 bits and get the upper half
// (3) use `vreinterpretq_s8_u8` to interpret the vector as int8

const uint8x16_t mask_low4bit = vdupq_n_u8(0xf);
uint8x16_t lower_half = vandq_u8(w0, mask_low4bit);
uint8x16_t upper_half = vshrq_n_u8(w0, 4);

int8x16_t w0_lower_s8 = reinterpretq_s8_u8(lower_half);
int8x16_t w0_upper_s8 = reinterpretq_s8_u8(upper_half);

// TODO: apply zero_point to weights and convert the range from
(0, 15) to (-8, 7)
// Hint: using `vsubq_s8` to the lower-half and upper-half
vectors of weights
const int8x16_t offsets = vdupq_n_s8(8);
w0_lower_s8 = vsubq_s8(w0_lower_s8, offsets);
w0_upper_s8 = vsubq_s8(w0_upper_s8, offsets);

// load 32 8-bit activation
const int8x16_t a0 = vld1q_s8(a_start);
const int8x16_t a1 = vld1q_s8(a_start + 16);
a_start += 32;

// TODO: perform dot product and store the result into the
intermediate sum, int_sum0
// Hint: use `vdotq_s32` to compute sumv0 = a0 * lower-half
weights + a1 * upper-half weights
```

```

        // int32x4 vector to store intermediate sum
        int32x4_t int_sum0 = vdupq_n_s32(0);
        int_sum0 = vdotq_s32(int_sum0, w0_lower_s8, a0);
        int_sum0 = vdotq_s32(int_sum0, w0_upper_s8, a1);

        float s_0 = *s_a++ * *s_w++;
        sumv0 = vmlaq_n_f32(sumv0, vcvtq_f32_s32(int_sum0), s_0);
    }

```

- b. How does the performance in GOPs, achieved through SIMD programming on your computer, compare to the reference implementation? Please explain the performance difference. (5pt)

```

----- Sanity check of reference implementation: Passed! -----
Section, Total time(ms), Average time(ms), Count, GOPs
reference, 3077.961914, 307.795990, 10, 0.851680
----- Sanity check of simd_programming implementation: Passed! -----
Section, Total time(ms), Average time(ms), Count, GOPs
simd_programming, 1434.850952, 143.485001, 10, 1.826977

```

The SIMD has an improvement of about 2.1451x. And used about less than half the time consumption. This is because the Single Instruction Multiple Data method is a parallel processing paradigm that applies a single instruction to multiple data elements simultaneously. With it, we can increase computational throughput and speed and improve energy efficiency.

4. Multithreading with Loop Unrolling (20pt): Please fill in the starter code in *kernel/template/multithreading_loop_unrolling.cc* to implement multithreading and loop unrolling and run the `./evaluate.sh multithreading_loop_unrolling` to evaluate the performance improvement.`
- a. Please copy and paste your implementation in *kernel/template/multithreading_loop_unrolling.cc*: (15pt)

```

        int intermediate_sum0 = 0, intermediate_sum1 = 0,
intermediate_sum2 = 0, intermediate_sum3 = 0;
        for (int qj = 0; qj < 16; qj++) {
            // TODO: decode a packed byte into two int8 in the range of
(-8, 7)

            uint8_t packed_int4_0 = w0_int4[qj];
            uint8_t packed_int4_1 = w1_int4[qj];
            uint8_t packed_int4_2 = w2_int4[qj];
            uint8_t packed_int4_3 = w3_int4[qj];

            signed char w_de_0_0 = (packed_int4_0 & 0x0F) - 8.0;
            signed char w_de_0_16 = (packed_int4_0 >> 4) - 8.0;
            signed char w_de_1_0 = (packed_int4_1 & 0x0F) - 8.0;
            signed char w_de_1_16 = (packed_int4_1 >> 4) - 8.0;
            signed char w_de_2_0 = (packed_int4_2 & 0x0F) - 8.0;
            signed char w_de_2_16 = (packed_int4_2 >> 4) - 8.0;
            signed char w_de_3_0 = (packed_int4_3 & 0x0F) - 8.0;
            signed char w_de_3_16 = (packed_int4_3 >> 4) - 8.0;

            // TODO: int8 multiply and accumulate operation
            intermediate_sum0 += a_int8[qj] * w_de_0_0;
            intermediate_sum0 += a_int8[qj+16] * w_de_0_16;
            intermediate_sum1 += a_int8[qj] * w_de_1_0;
            intermediate_sum1 += a_int8[qj+16] * w_de_1_16;
            intermediate_sum2 += a_int8[qj] * w_de_2_0;
            intermediate_sum2 += a_int8[qj+16] * w_de_2_16;
            intermediate_sum3 += a_int8[qj] * w_de_3_0;
            intermediate_sum3 += a_int8[qj+16] * w_de_3_16;
        }
... ..
namespace matmul {
void MatmulOperator::mat_mul_multithreading_loop_unrolling(struct matmul_params
*params) {
... ..
    // Thread creation
    for (int j = 0; j < num_thread; j++) {
        threads_args[j].start = j * (n / num_thread);
        threads_args[j].end = (j == num_thread - 1) ? n : (j + 1) * (n /
num_thread); // Ensure last thread covers all remaining
        threads_args[j].params = params; // Pass the params pointer directly
        pthread_create(&thread_pool[j], NULL,
multithreading_loop_unrolling_worker_func, &threads_args[j]);
    }
    // Join threads
    for (int j = 0; j < num_thread; j++)

```

```

{
    pthread_join(thread_pool[j], NULL);
}
... ..

```

- b. How does the performance in GOPs, achieved through multithreading and loop unrolling on your computer, compare to the reference implementation? Please explain the performance difference. (5pt)

----- Sanity check of reference implementation: Passed! -----

Section, Total time(ms), Average time(ms), Count, GOPs
reference, 3077.961914, 307.795990, 10, 0.851680

----- Sanity check of multithreading_loop_unrolling implementation: Passed!

Section, Total time(ms), Average time(ms), Count, GOPs
multithreading_loop_unrolling, 596.890991, 59.688999, 10, 4.391824

The multithreading and loop unrolling combination method has reached so far the most significant optimization on GOPs and time. It has a 5.1566 times of GOPs compared to the reference implementation, and uses only 0.1939239 times of the original average time.

5. Combination of All Techniques (20pt): Please fill in the starter code in *kernel/template/all_techniques.cc* to implement all techniques above and run the `./evaluate.sh all_techniques` to evaluate the performance improvement.

- a. Please copy and paste your implementation in *kernel/template/all_techniques.cc*: (15pt)

- b. How does the performance in GOPs, achieved through all optimization techniques on your computer, compare to the reference implementation? Please explain the performance difference. (5pt)

----- Sanity check of reference implementation: Passed! -----

Section, Total time(ms), Average time(ms), Count, GOPs
reference, 3077.961914, 307.795990, 10, 0.851680

----- Sanity check of all_techniques implementation: Passed! -----

Section, Total time(ms), Average time(ms), Count, GOPs
all_techniques, 152.794998, 15.279000, 10, 17.156582

Using all optimization techniques, we have an improvement on GOPs by 20.144x. And the total time consumption decreased by 0.0496x.